



**FACULTAD
DE INGENIERIA**

Universidad de Buenos Aires

75.29/95.06/TB024 Teoría de Algoritmos

Trabajo Práctico N°1

Grupo N° 4

Apellido, Nombres	Email	Padrón
Tania Friedenberger	tfriedenberger@fi.uba.ar	108823
Marco Tosi	mtosi@fi.uba.ar	107237
Payero Juan Ignacio	jpayero@fi.uba.ar	113195
Bloise Julieta	jbloise@fi.uba.ar	98592
Guillermo Hahn	ghahn@fi.uba.ar	112761

Fecha de 1er entrega:	26/04/2026
------------------------------	------------

Observaciones:

Índice

Índice	1
Problema 1: División y Conquista	2
1. Requerimientos del Algoritmo	2
2. Explicación del código	2
3. Análisis de Complejidad Asintótica	2
4. Análisis de Complejidad del Algoritmo Propuesto	3
5. Pseudocódigo	4
6. Gráficos	5
7. Conclusión	6
Problema 2: Greedy	7
1. ¿Qué problema resuelve el código encontrado?	7
2. ¿Qué algoritmo conocido replica dicho código?	7
3. ¿Cuál es el uso de las principales estructuras de datos del programa?	7
4. ¿Por qué se puede considerar que es un algoritmo Greedy? ¿Cuál es la regla que se aplica?	7
5. Complejidad temporal y espacial. ¿Cuál es el orden de complejidad temporal y espacial del programa?	8
6. Seguimiento de ejemplo	8
7. Pseudocódigo	10
8. Traducción a python	10
Problema 3: Backtracking	11
1. Supuestos	11
2. Estructuras de datos:	11
3. Pseudocódigo	12
4. Seguimiento:	12
5. Análisis de Complejidad:	14
a. Complejidad temporal	14
b. Complejidad espacial	14
6. Sets de Datos y Tiempos de Ejecución	15
7. Correspondencia con la complejidad teórica	16
Problema 4: Programación Dinámica	17
1. Supuestos	17
2. Diseño	17
3. Seguimiento	19
4. Complejidades	20
5. Informe de Resultados	21
6. Análisis de los Tiempos de Ejecución	22
7. Conclusión	22

Problema 1: *División y Conquista*

1. Requerimientos del Algoritmo

- El algoritmo espera una lista de enteros comparables "int" ingresados como parámetro.
- El algoritmo espera que si o si haya una y solo una moneda de menor peso, y que las demás monedas tengan exactamente el mismo peso
- El algoritmo espera una lista de pesos positivos mayores a cero

2. Explicación del código

El algoritmo que resuelve este problema cuenta con una función auxiliar que actúa como un pesaje para dos bolsas de monedas distintas, devolviendo números mayores a 1 si la primer bolsa es más pesada que la segunda, menor a 1 si es al revés, y devolviendo 0 si las bolsas pesan lo mismo

La función en cuestión recibe por parámetro una lista con los pesos de más monedas de la bolsa, las cuales cuentan todas con el mismo peso excepto por una, la cual tiene un valor de peso menor. El peso de las monedas es un tipo de datos comparable, en este caso tipo "int" (entero)

La función cuenta con un caso base inicialmente, al cual se accede si llegamos a tener una bolsa con una sola moneda, y dado que se asume que la moneda falsa es una y siempre se encuentra en la bolsa, cumple de esa manera la condición del enunciado

Luego del caso base, la bolsa de monedas se divide a la mitad (bolsa izquierda y bolsa derecha), donde tenemos dos casos posibles, una bolsa con una mayor cantidad de monedas que la otra, o ambas bolsas con la misma cantidad de monedas.

En el primer caso, se toma la moneda del medio y no se tomará para el pesaje de bolsas (quedando así dos bolsas con igual cantidad de monedas). Se realiza una comparación entre las bolsas y si resultan tener igual peso, significa que es la moneda del medio la más falsa y por ende retornamos su índice en la lista de monedas

En el segundo caso, con dos bolsas de mismo tamaño, se realiza el pesaje y si la bolsa izquierda pesa menos, significa que la moneda falsa está en esa bolsa, si no ocurre eso, se entonces se encuentra en la bolsa derecha (pues se asume que si o si hay una moneda más liviana que el resto). En ambos casos se busca en su respectiva bolsa, descartando la otra.

Este proceso se repite hasta llegar al caso base, o cumplir la condición de tener la moneda falsa en el medio de la lista. Finalmente se retorna la posición en la lista donde se encuentra la misma.

3. Análisis de Complejidad Asintótica

El análisis de complejidad nos permite determinar cómo crece el tiempo de ejecución de un algoritmo a medida que aumenta el tamaño de la entrada. Este crecimiento suele expresarse mediante notación asintótica. (Cormen et al. 2022)

4. Análisis de Complejidad del Algoritmo Propuesto

Función Auxiliar (pesar_grupos_de_monedas())

La función en cuestión realiza una sola operación, donde ejecuta la función sum() dos veces sobre dos listas respectivamente. Esta función tiene costo $O(N)$ siendo N la cantidad de elementos de la lista pasada por parámetro. Dado que esto se realiza para dos listas que representan dos mitades, podemos decir que la complejidad de la función es $O(n)$

Siendo n la cantidad de elementos de la lista de monedas al momento de llamar la función

Función Principal (encontrar_moneda_mas_liviana_dyc())

Al analizar el algoritmo, tenemos una serie de instrucciones, que entre las cuales encontramos el caso base:

```
cantidad = len(monedas)
if cantidad == 1:
    return 0

mitad = cantidad // 2
if cantidad % 2 != 0:
    mitad_izquierda = monedas[:mitad]
    mitad_derecha = monedas[mitad + 1 :]
```

Estas operaciones que incluyen condicionales “if” y operaciones aritméticas y lógicas, se realizan en tiempo constante $O(1)$.

Luego se ejecuta la función de pesaje que como ya vimos tiene un costo de $O(n)$ siendo n la cantidad de elementos de las dos listas a pesar.

```
if pesar_grupos_de_monedas(mitad_izquierda, mitad_derecha) == 0:
    return mitad
```

Aparte de esto, la instrucción “if” y el retorno tienen complejidad constante $O(1)$

Luego siguen una serie de asignaciones que cuentan con una complejidad constante $O(1)$

```
else:
    mitad_izquierda = monedas[:mitad]
    mitad_derecha = monedas[mitad:]
```

Finalmente contamos un condicional que llama a la función de pesaje (complejidad $O(n)$) y en base al resultado de esta función, se realiza una llamada recursiva (solo una por ciclo recursivo) a la función principal.

De esta forma la complejidad INTERNA de la función es

$$O(2n) \simeq O(n)$$

Esto nos permite formar una fórmula de recursión para este algoritmo en específico:

$$T(n) = A T\left(\frac{n}{B}\right) + O(n^C)$$

Donde:

- A. es la cantidad de llamadas recursivas que se ejecutan por llamada recursiva
- B. es la cantidad de partes proporcionales en las que se divide el problema
- C. Representa la complejidad interna de cada llamada recursiva

En este caso tenemos que: $A=1$ pues solo se hace una llamada elegida mediante un condicional "if"; $B=2$ pues el problema se divide en 2 partes iguales en cada llamada; $C=1$ pues el costo interno de cada llamada es $O(n)$ dado que las llamadas a la función de pesaje son de complejidad lineal y las demás operaciones son de tiempo constante.

Análisis Según el Teorema Maestro (Cormen et al., 2022):

Según lo calculado anteriormente, la función de recursión del algoritmo nos queda

$$T(n) = 1 T\left(\frac{n}{2}\right) + O(n^1)$$

Por esto mismo, al realizar la cuenta: $\log_B A = \log_2 1 = 0$.

Luego comparamos con el valor de C y dado que $\log_2 1 < C$ incurrimos en el tercer caso del teorema maestro de modo que la complejidad teórica del algoritmo es:

$$O(n^C) = O(n)$$

5. Pseudocódigo

función encontrar_moneda_mas_liviana_dyc(monedas):

 //caso base

 if la cantidad de monedas es 1:

 esa única moneda es la falsa, retornamos

 //caso recursivo

 calculamos el medio de la lista de monedas

 if la cantidad de monedas es impar:

 separamos la lista en dos mitades, sin incluir el medio

 pesamos ambas mitades

 if las mitades pesan lo mismo

 el medio es la moneda falsa, retornamos

 sino:

 separamos las monedas en dos mitades iguales

 pesamos ambas mitades

 if la mitad izquierda es más liviana:

 buscamos en la mitad izquierda la moneda falsa

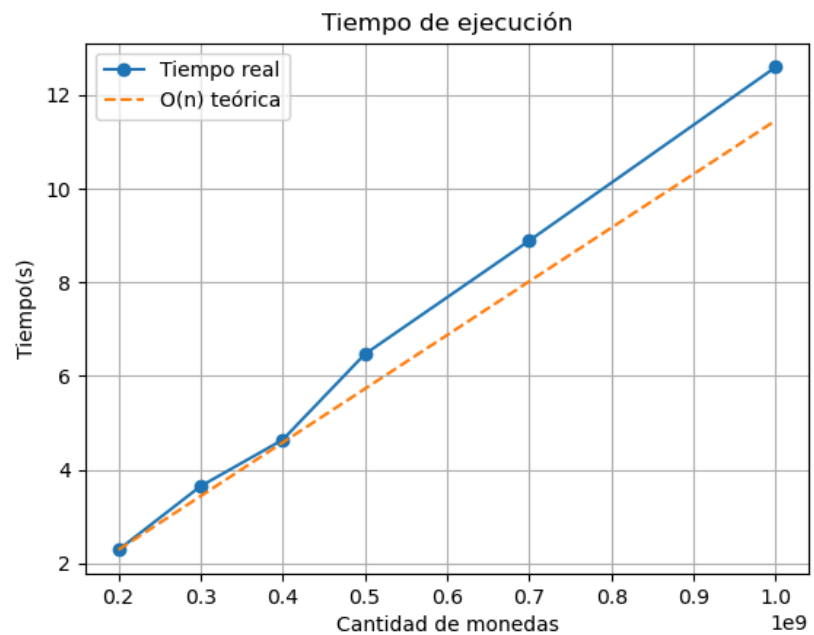
 sino

 buscamos en la mitad derecha la moneda falsa

6. Gráficos

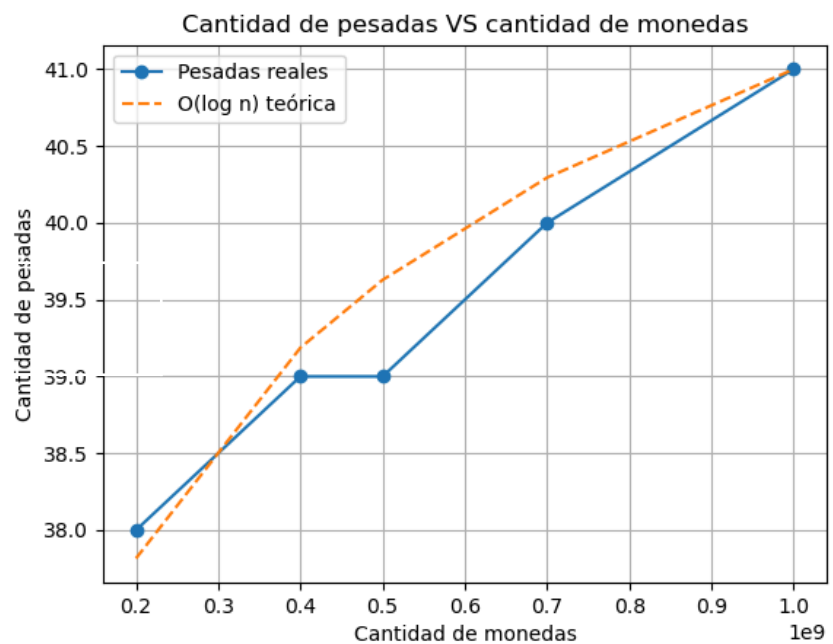
Cantidad De Monedas VS Tiempo de Ejecución

Cantidades de monedas representadas en el orden de los 1,000,000,000



Cantidad De Monedas VS Cantidad de Pesadas

Cantidades de monedas representadas en el orden de los 1,000,000,000



7. Conclusión

¿Es correcto vincular el tiempo de ejecución con la cantidad de pesadas?:

No, en si no es correcto relacionar tiempo de ejecución con cantidad de pesadas debido a la forma en que crecen estos valores.

Por un lado, la cantidad de pesadas crece de forma logarítmica ya que en cada iteración la cantidad de elementos se reduce a la mitad.

Por otro lado, el tiempo de ejecución crece de forma lineal, cosa que fue demostrada teóricamente y comprobada con el gráfico.

En base al análisis de los distintos gráficos, notamos que los tiempos de ejecución así como la cantidad de pesadas coinciden con su proyección teórica, y, en complemento con la pregunta anterior, podemos concluir que el principal impedimento para que la cantidad de pesadas y el tiempo de ejecución estén directamente relacionados es el costo mismo de cada pesaje.

Esto es porque el algoritmo implementado usa una función con costo $O(n)$ (siendo n la cantidad total de monedas de las dos bolsas/listas que se van a pesar en ese momento) debido al uso de la función `sum()`.

Cosa que en un modelo teórico ideal (o en el mundo real con una balanza analogica), donde los pesajes se realizan mediante una balanza de costo constante, el tiempo de ejecución sí sería proporcional a la cantidad de pesadas, lográndose una complejidad $O(\log n)$, pero que en la práctica no se cumple, impedimento que genera costos de tiempo más altos.

Problema 2: *Greedy*

1. ¿Qué problema resuelve el código encontrado?

El problema que resuelve el algoritmo es el problema de “**Camino Mínimo**” en un grafo pesado y no dirigido. Dado un vértice “origen” se calcula la distancia mínima para llegar desde el origen a los demás vértices.

2. ¿Qué algoritmo conocido réplica dicho código?

El código implementa el algoritmo de **Dijkstra**.

3. ¿Cuál es el uso de las principales estructuras de datos del programa?

El código tiene 3 estructuras en memoria:

- DIM COST (N,N) = **Matriz de adyacencias**($O(n^2)$). Sirve para representar el grafo, almacena los pesos de las aristas(peso 15000 para representar infinito).
- DIM DIST (N) = **Arreglo de distancias**($O(n)$). Almacena la distancia mínima conocida desde el vértice “origen” hasta el vértice i .
- DIM SOL (N) = **Arreglo de vértices visitados**($O(n)$). Almacena valores booleanos para saber qué nodos tienen la distancia mínima definida.

4. ¿Por qué se puede considerar que es un algoritmo Greedy? ¿Cuál es la regla que se aplica?

Se considera Greedy porque construye la solución tomando en cada iteración la “mejor decisión” en ese momento(**óptimo local**), esperando a que nos lleve a la mejor solución general(**óptimo global**). Se encuentra la solución óptima si y sólo si el grafo no tiene aristas con pesos negativos.

La regla que se aplica es: “De los nodos no visitados elegir el que tenga menor distancia acumulada”. Al elegir el nodo U con la menor distancia actual, asumimos que es imposible encontrar un camino más corto hasta U. Como todos los pesos son positivos, cualquier otro camino hasta U va a tener una distancia mayor o igual a la previamente calculada porque ir por otro camino es sumarle más aristas lo que aumenta el costo. De esta manera, podemos decir que la decisión local asegura el óptimo global. Se puede ver como se encuentra el óptimo local en estas 3 líneas de código.

```
1040 FOR J=1 TO N
1050 IF DIST(J) ≤ U AND SOL(J)=0 THEN U=J
1060 NEXT J
```

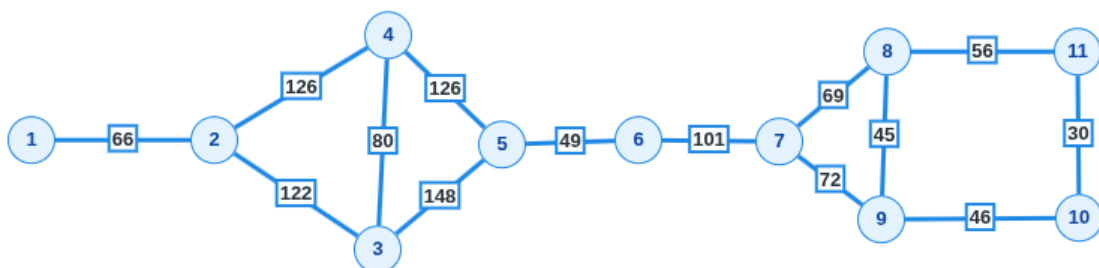

5. Complejidad temporal y espacial. ¿Cuál es el orden de complejidad temporal y espacial del programa?

- Datos a tener en cuenta:** cantidad de vértices = N (en el código).
- Complejidad Temporal:** El bucle principal (línea 1020) itera $N-1$ veces $\approx V$. Dentro del bucle, hay dos operaciones:
 - búsqueda lineal para encontrar el nodo con la distancia mínima (líneas 1040-1060, costo $O(N)$)
 - iteración sobre todos los vecinos para ver si hay un camino más corto (líneas 1080-1100, costo $O(N)$).
 - Sumando las complejidades nos queda: $N * (N + N) = 2N^2$
 - La complejidad final es:** $O(N^2)$

```
1020 FOR I=1 TO N-1
1030 U=15000
1040 FOR J=1 TO N
1050 IF DIST(J) ≤ U AND SOL(J)=0 THEN U=J
1060 NEXT J
1070 SOL(U)=1
1080 FOR J=1 TO N
1090 IF DIST(J) ≥ (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
1100 NEXT J
```

- Complejidad espacial:** La complejidad espacial es de $O(N^2)$. El espacio en memoria es dominado por la matriz de adyacencia **COST** que requiere $N*N$ espacios para almacenar las relaciones entre los vértices. Los arreglos **DIST** y **SOL** ocupan $O(N)$ por lo que quedan desplazados por el término de mayor grado.

6. Seguimiento de ejemplo



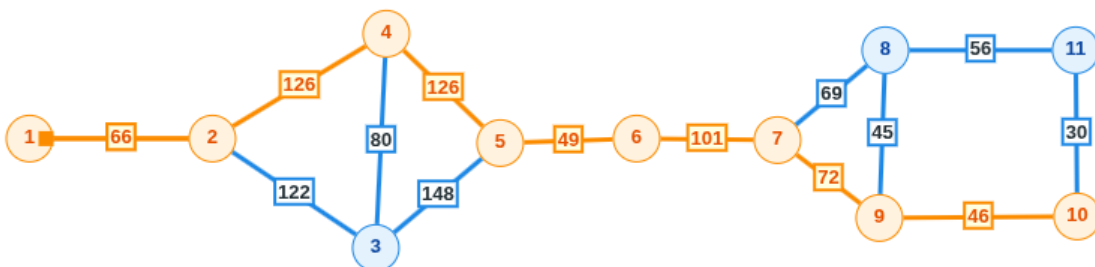
- Del nodo 1 al nodo 2 (Único camino) | distancia_acumulada = 66
- Del nodo 2 al nodo 3 (Regla Greedy: $188 < 192$) | distancia_acumulada = 188
- Del nodo 2 al nodo 4 (Siguiendo mínimo en la lista) | distancia_acumulada = 192

PUNTO CRÍTICO: Camino más corto al Nodo 5

- Del nodo 3 al nodo 5 (cálculo inicial) | $\text{distancia_tentativa} = 188 + 148 = 336$
- Del nodo 4 al nodo 5 (reducción: $318 < 336$) | $\text{distancia_acumulada} = 192 + 126 = 318$
- El camino por el nodo 4 parece ser más largo pero resultó ser el más óptimo para llegar al Nodo 5
- Del nodo 5 al nodo 6 (Único camino) | $\text{distancia_acumulada} = 318 + 49 = 367$
- Del nodo 6 al nodo 7 (Único camino) | $\text{distancia_acumulada} = 367 + 101 = 468$

PUNTO CRÍTICO: Elección hacia el vértice final

- Del nodo 7 al nodo 8 (Regla Greedy: $537 < 540$) | $\text{distancia_acumulada} = 468 + 69 = 537$
- Del nodo 7 al nodo 9 (Siguiendo mínimo) | $\text{distancia_acumulada} = 468 + 72 = 540$
- El algoritmo prioriza el camino al nodo 8. La diferencia de 3 unidades es la que define el camino más corto hacia el vértice final.
- Del nodo 9 al nodo 10 (Siguiendo mínimo) | $\text{distancia_acumulada} = 540 + 46 = 586$
- Del nodo 8 al nodo 11 (Cerrando el circuito) | $\text{distancia_acumulada} = 537 + 56 = 593$



7. Pseudocódigo

```
# Grafo G
# Vertice origen V
def Dijkstra(G, V)
    Para cada vertice i en G:
        distancia[i] = inf
        visitado[i] = FALSO
    distancia[V] = 0

    Mientras queden vertices sin visitar:
        U = vertice no visitado con el valor mínimo en distancia[]

        Si distancia[U] = inf:
            break // Nodos restantes inalcanzables

        visitado[U] = VERDADERO // Se marca óptimo local

        Para cada vertice adyacente J de U:
            Si visitado[J] = FALSO:
                nuevo_costo = distancia[U] + peso(U, J)
                Si nuevo_costo < distancia[J]:
                    distancia[J] = nuevo_costo

    return distancia
```

8. Traducción a python

La traduccion esta en el archivo **greedy.py**

Problema 3: *Backtracking*

1. Supuestos

El algoritmo desarrollado opera bajo los siguientes supuestos:

- El laberinto es una grilla rectangular de m filas por n columnas.
- Las celdas pueden ser de cuatro tipos: 'X' (pared), ' ' (camino libre), 'E' (posición inicial de la aspiradora) y 'S' (salida).
- Existe exactamente una celda 'E' y una celda 'S' en el laberinto.
- Cualquier posición que se encuentre fuera de los límites de la grilla se trata como pared.
- La aspiradora desconoce la estructura global del laberinto; únicamente puede ejecutar las tres operaciones definidas por el enunciado.
- Se prueban las cuatro orientaciones cardinales iniciales (N, E, S, O). Se adopta la primera orientación que conduzca a una solución.
- El camino resultante es simple: no contiene posiciones repetidas.
- No hay “islas” o columnas para no tener que recorrer nuevamente posiciones ya pasadas, de todas formas el algoritmo funciona con eso.

2. Estructuras de datos:

Estructura	Tipo	Propósito
<code>maze</code>	Lista de listas de <code>char</code>	Representa el laberinto.
<code>path</code>	Lista de tuplas (<code>fila</code> , <code>col</code>)	Registra el camino físico recorrido en la rama activa. Se construye con <code>append</code> y se deshace con <code>pop</code> al hacer backtrack.
<code>path_set</code>	Conjunto de tuplas (<code>fila</code> , <code>col</code>)	Espejo de <code>path</code> para consulta $O(1)$. Garantiza que el camino activo sea simple: impide pisar posiciones ya incluidas en el camino actual.
<code>visited</code>	Conjunto de tuplas (<code>fila</code> , <code>col</code> , <code>dir</code>)	Registra los estados explorados en la rama activa. Se agrega al entrar y se remueve al hacer backtrack, garantizando que no haya ciclos dentro de una misma rama pero permitiendo la re-exploración en ramas distintas.

El rol de visited es local a la rama: al deshacerse con el backtrack, un mismo estado (pos, dir) puede ser re-explorado en otra rama donde path_set tenga distinto contenido. Esto garantiza la completitud de la exploración sin repetir trabajo dentro de una misma rama.

3. Pseudocódigo

```
resolver(pos, dir, esAvance):
    estado ← (pos, dir)
    si estado ∈ visited → retornar FALSO          // ciclo en esta rama
    visited.add(estado)

    si esAvance:
        si pos ∈ path_set → visited.remove(estado); retornar FALSO
        si laberinto[pos] == SALIDA:
            camino.append(pos); retornar VERDADERO
            camino.append(pos); path_set.add(pos)
        sino: // es un giro, no un avance
            registrar giro en log

    adelante ← verAdelante(pos, dir)             // Operación 1
    si adelante ≠ PARED:
        nuevaPos ← avanzar(pos, dir)             // Operación 2
        si resolver(nuevaPos, dir, True):
            visited.remove(estado); retornar VERDADERO

    nuevaDir ← girarIzquierda(dir)               // Operación 3
    si resolver(pos, nuevaDir, False):
        visited.remove(estado); retornar VERDADERO

    si esAvance:                                // BACKTRACK
        camino.pop(); path_set.remove(pos)
        visited.remove(estado)                   // liberar para otras ramas
    retornar FALSO
```

La función se invoca inicialmente con esAvance = True y la posición de la celda 'E'. Se prueban las cuatro orientaciones iniciales; se detiene al encontrar la primera solución.

El algoritmo distingue dos tipos de llamadas recursivas:

- **Avance** (esAvance = True): la aspiradora se desplaza a una nueva celda. La posición se incorpora a path y path_set. Al hacer backtrack, se remueve de ambas estructuras.
- **Giro** (esAvance = False): la aspiradora cambia de dirección sin moverse. No modifica path ni path_set. Sirve para explorar nuevas orientaciones desde la misma posición.

4. Seguimiento:

```
=====
Laberinto: ej1.txt (7×6) [42 celdas]
Inicio: (6, 1) Salida: (1, 5)
```

```
Avance  1 | prof= 1 | (6, 1) dir=N
        Ve adelante (N): CAMINO
Avance  2 | prof= 2 | (5, 1) dir=N
        Ve adelante (N): CAMINO
Avance  3 | prof= 3 | (4, 1) dir=N
        Ve adelante (N): CAMINO
Avance  4 | prof= 4 | (3, 1) dir=N
```

Ve adelante (N): CAMINO
Avance 5 | prof= 5 | (2, 1) dir=N
Ve adelante (N): CAMINO
Avance 6 | prof= 6 | (1, 1) dir=N
Ve adelante (N): PARED
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (2, 1): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): CAMINO
Avance 7 | prof= 7 | (1, 2) dir=E
Ve adelante (E): PARED
Gira → ahora mira N
Ve adelante (N): PARED
Gira → ahora mira O
Ve adelante (O): CAMINO
Gira → ahora mira S
Ve adelante (S): PARED
———— BACKTRACK: abandona (1, 2), vuelve a (1, 1) (prof=6)
———— BACKTRACK: abandona (1, 1), vuelve a (2, 1) (prof=5)
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (3, 1): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): PARED
———— BACKTRACK: abandona (2, 1), vuelve a (3, 1) (prof=4)
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (4, 1): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): CAMINO
Avance 8 | prof= 5 | (3, 2) dir=E
Ve adelante (E): PARED
Gira → ahora mira N
Ve adelante (N): PARED
Gira → ahora mira O
Ve adelante (O): CAMINO
Gira → ahora mira S
Ve adelante (S): PARED
———— BACKTRACK: abandona (3, 2), vuelve a (3, 1) (prof=4)
———— BACKTRACK: abandona (3, 1), vuelve a (4, 1) (prof=3)
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (5, 1): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): PARED
———— BACKTRACK: abandona (4, 1), vuelve a (5, 1) (prof=2)
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (6, 1): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): CAMINO
Avance 9 | prof= 3 | (5, 2) dir=E
Ve adelante (E): CAMINO
Avance 10 | prof= 4 | (5, 3) dir=E
Ve adelante (E): CAMINO
Avance 11 | prof= 5 | (5, 4) dir=E

```
Ve adelante (E): PARED
Gira → ahora mira N
Ve adelante (N): CAMINO
Avance 12 | prof= 6 | (4, 4) dir=N
Ve adelante (N): CAMINO
Avance 13 | prof= 7 | (3, 4) dir=N
Ve adelante (N): CAMINO
Avance 14 | prof= 8 | (2, 4) dir=N
Ve adelante (N): CAMINO
Avance 15 | prof= 9 | (1, 4) dir=N
Ve adelante (N): PARED
Gira → ahora mira O
Ve adelante (O): PARED
Gira → ahora mira S
Ve adelante (S): CAMINO
[descarta (2, 4): ya en camino activo]
Gira → ahora mira E
Ve adelante (E): SALIDA
Avance 15 | prof= 10 | (1, 5) dir=E → SALIDA ✓
```

--- Resultado ---

Solucion: 10 celdas (0.000067 s)

Camino: (6, 1) → (5, 1) → (5, 2) → (5, 3) → (5, 4) → (4, 4) → (3, 4) → (2, 4) → (1, 4) → (1, 5)

5. Análisis de Complejidad:

a. Complejidad temporal

El espacio de estados del algoritmo está definido por la tupla (fila, columna, dirección). Dado un laberinto de m filas y n columnas, el número total de estados posibles es:

$$|\text{Estados}| = m \times n \times 4$$

La estructura `visited`, local a cada rama, garantiza que ningún estado (pos, dir) sea procesado más de una vez dentro de una misma rama de ejecución. Dado que cada estado realiza trabajo constante (una llamada a `look_ahead` y a lo sumo dos llamadas recursivas), la complejidad temporal es:

$$T(m,n) = O(m \times n \times 4) = O(m \times n)$$

Esta cota se mantiene porque `path_set` impide que el camino activo crezca más allá de $m \times n$ posiciones, lo que a su vez acota la profundidad de la recursión y, por ende, el número total de estados procesados por rama.

b. Complejidad espacial

Las estructuras auxiliares consumen espacio proporcional al número de celdas del laberinto:

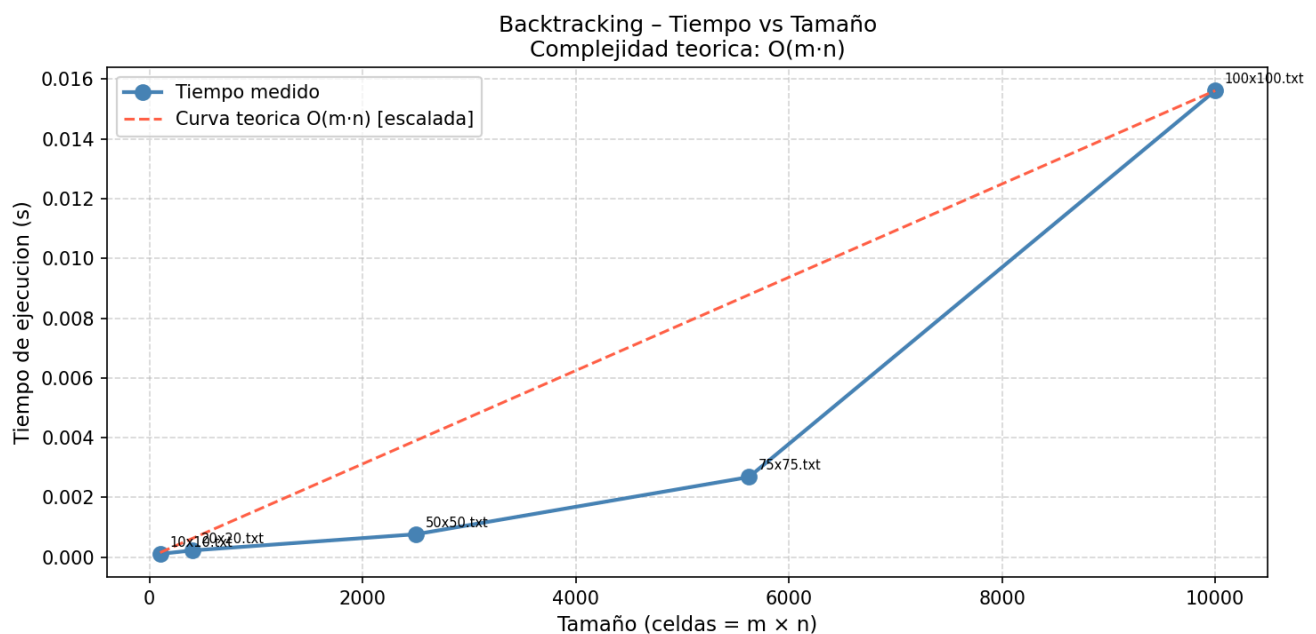
- `path` y `path_set`: en el peor caso contienen todas las celdas libres del laberinto → $O(m \times n)$.
- `visited`: en el peor caso registra todos los estados explorados en la rama activa → $O(m \times n)$.
- Pila de recursión: la profundidad máxima está acotada por el tamaño del camino activo → $O(m \times n)$.

La complejidad espacial total es $S(m,n) = O(m \times n)$

6. Sets de Datos y Tiempos de Ejecución

Se diseñaron sets de datos de distintos tamaños para medir el comportamiento temporal del algoritmo. Cada laberinto fue generado de forma que garantizara la existencia de al menos un camino entre **E** y **S**, con estructuras de paredes variadas para reflejar escenarios de exploración representativos.

Archivo	Dimensiones	Celdas totales	Tiempo (s)
10x10.txt	10 × 10	100	0.000113
20x20.txt	20 × 20	400	0.000227
50x50.txt	50 × 50	2.500	0.000770
75x75.txt	75 x 75	5625	0.002683
100x100.txt	100 × 100	10.000	0.015612



7. Correspondencia con la complejidad teórica

La complejidad teórica establece que el número de operaciones escala linealmente con el tamaño del laberinto. Los tiempos empíricos confirman esta tendencia: al duplicar las dimensiones del laberinto (y por tanto cuadruplicar las celdas), el tiempo de ejecución se incrementa aproximadamente en un factor de cuatro, coherente con $O(m \times n)$.

Cabe destacar que el tiempo efectivo depende también de la densidad de caminos libres y de la posición relativa de **E** y **S**: laberintos con mayor cantidad de celdas libres implican más estados a explorar, mientras que laberintos donde la salida se encuentra tempranamente en el árbol de exploración pueden resolverse en tiempo sublineal respecto del peor caso teórico.

Problema 4: *Programación Dinámica*

1. Supuestos

- **Mayúsculas/Minúsculas:** se asume que se usa un texto normalizado ("A" y "a" se toma como letras distintas)
- **Memoria:** se asume que se tiene una memoria suficiente para tener una matriz de $N \times N$
- El algoritmo no considera espacios ni signos de puntuación como separadores.
- Todo **carácter individual es un palíndromo**, por lo que siempre existe al menos una partición válida (palíndromos de longitud 1).

2. Diseño

a. Ecuación de recurrencia

- Tabla de palíndromos. Sea **es_pal[i][j]** una matriz booleana que indica si las subcadenas(s) desde j hasta i son palindromas.

$$\text{es_pal}[i][j] = \begin{cases} \text{True} & \text{Si } j = i \\ s[j] == s[i] & \text{Si } i = j + 1 \\ s[j] == s[i] \text{ Y es_pal}[j+1][i-1] & \text{Si } i > j + 1 \end{cases}$$

- Se define el vector **dp[i]**, donde se guardará en el índice i el menor número de palíndromos desde el índice 0 hasta i.

$$\text{pd}[i] = \begin{cases} 1 & \text{Si cadena}[0...i] \text{ es palíndromo} \\ \min(\text{pd}[j-1] + 1) & \text{Si la cadena no es un palíndromo} \end{cases}$$

b. Subestructura Óptima y Subproblemas Superpuestos

- **Subestructura Óptima:** La solución óptima de $s[0..i]$ termina con el palíndromo $s[j..i]$, entonces la parte de $s[0..j-1]$ que aparece antes también debe ser óptima. Por tanto $\text{pd}[i] = \text{pd}[j-1] + 1$ donde j es el inicio del último fragmento en la solución óptima.

- **Subproblemas superpuestos:** Para calcular $pd[i]$, el algoritmo necesita múltiples veces los resultados de $pd[j - 1]$. Validar palíndromos largos requiere verificar repetidamente los mismos palíndromos más cortos ubicados en su interior.

c. Memoization

El algoritmo utiliza tabulación para evitar recalcular resultados:

- Una matriz **es_pal** donde se almacenan las comprobaciones de los palíndromos.
- Un arreglo **pd** que guarda el menor número de palíndromos

d. Pseudocódigo

```
def min_cant_palindromo(cadena):  
    n = len(cadena)  
    Si n == 0  
        retornar 0  
  
    es_pal = Matriz Booleana[n][n] iniciada en Falso  
    d = Arreglo Entero[n]  
  
    Para i = 0 hasta n-1:  
        es_pal[i][i] = Verdadero  
  
    Para L = 2 hasta n: // L = len(subcadena)  
        Para j = 0 hasta n - L:  
            i = j + L - 1  
            Si L == 2:  
                es_pal[j][i] = (cadena[j] == cadena[i])  
            Sino:  
                es_pal[j][i] = (cadena[j] == cadena[i]) Y es_pal[j+1][i-1]  
  
    Para i = 0 hasta n-1:  
        Si es_pal[0][i] == Verdadero:  
            pd[i] = 1 // Toda la subcadena hasta i es palíndromo  
        Sino:  
            pd[i] = infinito  
            Para j = 0 hasta i:  
                Si es_pal[j][i] == Verdadero:  
                    pd[i] = min(pd[i], pd[j-1] + 1)  
  
    Retornar pd[n-1]
```

e. Estructuras de datos utilizadas

- **Matriz de Booleanos(es_pal):** Para identificar palíndromos en tiempo constante en la segunda fase. Tamaño $O(n^2)$
- **Arreglo de Enteros(pd):** Para almacenar el acumulado mínimo de divisiones de los subproblemas. Tamaño $O(n)$

3. Seguimiento

Se usa la cadena “**ARACALACANA**” (n=11). Se muestran ambas fases del algoritmo

a. **Matriz es_pal[i][j]**

j \ i	A(0)	R(1)	A(2)	C(3)	A(4)	L(5)	A(6)	C(7)	A(8)	N(9)	A(10)
A(0)	T	F	T	F	F	F	F	F	F	F	F
R(1)	-	T	F	F	F	F	F	F	F	F	F
A(2)	-	-	T	F	T	F	F	F	T	F	F
C(3)	-	-	-	T	F	F	F	T	F	F	F
A(4)	-	-	-	-	T	F	T	F	F	F	F
L(5)	-	-	-	-	-	T	F	F	F	F	F
A(6)	-	-	-	-	-	-	T	F	T	F	F
C(7)	-	-	-	-	-	-	-	T	F	F	F
A(8)	-	-	-	-	-	-	-	-	T	F	T
N(9)	-	-	-	-	-	-	-	-	-	T	F
A(10)	-	-	-	-	-	-	-	-	-	-	T

- b. **Llenar el arreglo `pd[i]`:** Evaluamos letra por letra. Buscamos el mejor punto de corte `j` mirando los resultados de nuestra matriz.

i	Subcadena hasta i	¿es_pal[0][i] ?	Mejor (Corte)	j	pd[i] (Mínimo)	Último Formado	Palíndromo
0	A	T	0		1	A	
1	AR	F	1		2	R	
2	ARA	T	0		1	ARA	
3	ARAC	F	3		2	C	
4	ARACA	F	2		3	ACA	
5	ARACAL	F	5		4	L	
6	ARACALA	F	4		3	ALA	
7	ARACALAC	F	3		2	CALAC	
8	ARACALACA	F	2		3	ACALACA	
9	ARACALACAN	F	9		4	N	
10	ARACALACANA	F	8		3	ANA	

4. Complejidades

a. **Complejidad temporal:**

- Llenar la matriz **es_pal** que evalúa de $1 \dots n$, tiene 2 bucles anidados $\rightarrow O(n^2/2)$.
- Llenar el arreglo **pd** que evalúa de $0 \dots n-1$ y un bucle interno $0 \dots i \rightarrow O(n^2/2)$

Sumando las 2 complejidades nos da un **complejidad final:** $O(n^2)$

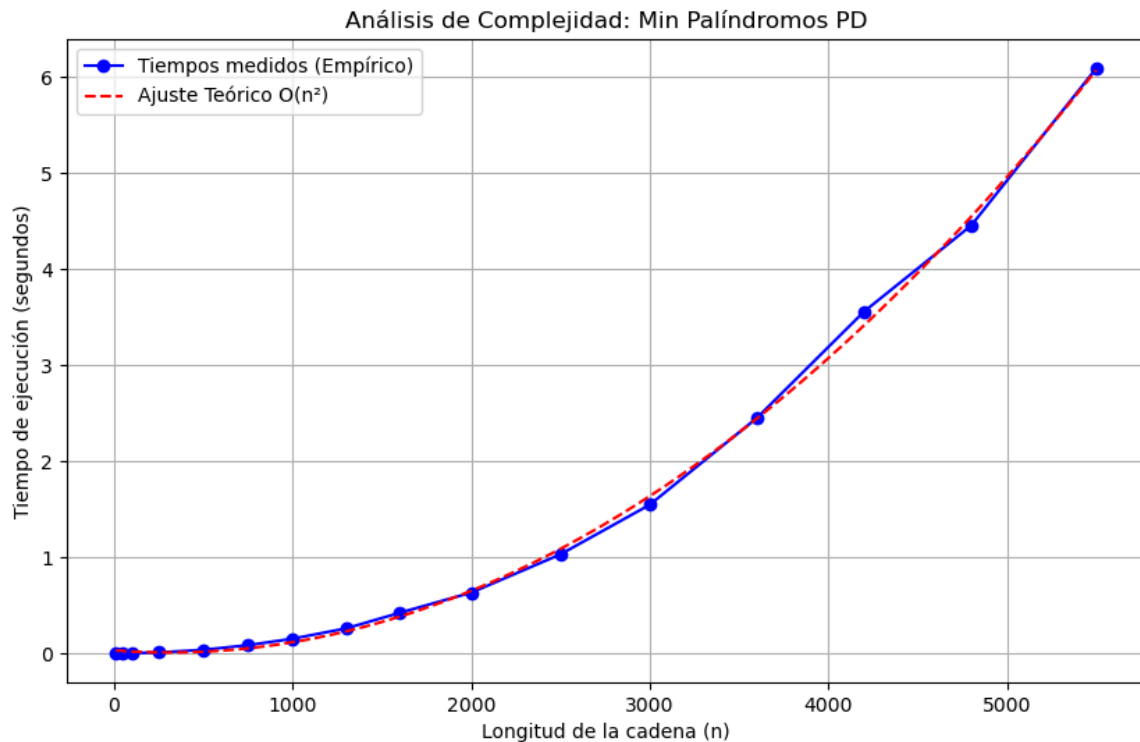
b. **Complejidad espacial:**

- Matriz **es_pal** de tamaño $n * n \rightarrow O(n^2)$ (condiciona mi complejidad teoría)
- Arreglo **pd** de tamaño $n \rightarrow O(n)$

Sumando las 2 complejidades nos da una **complejidad final:** $O(n^2)$

5. Informe de Resultados

Para evaluar el rendimiento del algoritmo de programación dinámica, se midieron los tiempos de ejecución utilizando cadenas de caracteres generadas aleatoriamente. La longitud de estas cadenas ($10 \leq n \leq 5500$).



```
RESULTADOS
N: 10 | Tamaño del palíndromo mas largo: 10 | Tiempo: 0.000031 segundos
N: 50 | Tamaño del palíndromo mas largo: 42 | Tiempo: 0.000471 segundos
N: 100 | Tamaño del palíndromo mas largo: 89 | Tiempo: 0.001199 segundos
N: 250 | Tamaño del palíndromo mas largo: 235 | Tiempo: 0.007620 segundos
N: 500 | Tamaño del palíndromo mas largo: 451 | Tiempo: 0.034820 segundos
N: 750 | Tamaño del palíndromo mas largo: 650 | Tiempo: 0.083233 segundos
N: 1000 | Tamaño del palíndromo mas largo: 888 | Tiempo: 0.150470 segundos
N: 1300 | Tamaño del palíndromo mas largo: 1171 | Tiempo: 0.258230 segundos
N: 1600 | Tamaño del palíndromo mas largo: 1437 | Tiempo: 0.420819 segundos
N: 2000 | Tamaño del palíndromo mas largo: 1775 | Tiempo: 0.630372 segundos
N: 2500 | Tamaño del palíndromo mas largo: 2241 | Tiempo: 1.029414 segundos
N: 3000 | Tamaño del palíndromo mas largo: 2667 | Tiempo: 1.545869 segundos
N: 3600 | Tamaño del palíndromo mas largo: 3203 | Tiempo: 2.449610 segundos
N: 4200 | Tamaño del palíndromo mas largo: 3741 | Tiempo: 3.556905 segundos
N: 4800 | Tamaño del palíndromo mas largo: 4286 | Tiempo: 4.452875 segundos
N: 5500 | Tamaño del palíndromo mas largo: 4915 | Tiempo: 6.088699 segundos
```

6. Análisis de los Tiempos de Ejecución

La observación de los datos recolectados revela un comportamiento de crecimiento exponencial. Se puede ver que a medida que el tamaño del problema aumentó, el costo se disparó.

Por ejemplo, al duplicar el tamaño de la entrada de $n = 1000 \rightarrow 0.15s$ a $n = 2000 \rightarrow 0.15s$, el tiempo se cuadruplica. Este patrón de aceleración se mantiene hasta el final de la muestra, donde procesar una cadena de $n = 4800$ toma aproximadamente 4.45s segundos, y un incremento de solo 700 caracteres adicionales (hasta $n = 5500$) eleva el tiempo total por encima de los 6.08s segundos.

Se puede ver que los tiempos medidos (azul) se entrelazan de manera casi perfecta con la curva de predicción teórica (roja). Las pequeñas variaciones observadas ($n = 4800$) son esperables a cualquier medición en un entorno real y responden a la gestión de memoria y procesos de fondo del sistema operativo.

7. Conclusión

El comportamiento empírico se corresponde de manera exacta con la complejidad determinada inicialmente. El análisis previo del pseudocódigo estableció una complejidad temporal de $O(n^2)$ debido al anidamiento de bucles requeridos tanto en la fase de identificación de palíndromos como en la fase de cálculo de cortes mínimos. La forma parabólica de los tiempos medidos y su parecido a la función de ajuste cuadrático demuestran empíricamente que el tiempo de ejecución del algoritmo crece en proporción al cuadrado del tamaño de la entrada $O(n^2)$.